

Salesforce in Teams

Teams SSO - Client Setup Guide

Silent sign-in: Microsoft Teams SSO + Salesforce OAuth 2.0 Token Exchange

Everything that must be configured in a new Microsoft tenant and Salesforce org, in the order it has to be done.

Verified end to end in the dev environment on 2026-09-26.
See the last section for what has NOT been verified.

2026-09-27

1. What you are setting up

Today the Salesforce in Teams tab signs each person in with a popup showing Salesforce's login page (the PKCE flow). This guide adds silent sign-in on top of it: when someone opens the tab, Teams hands the page a signed Microsoft Entra ID token for the person who is already signed in to Teams, Salesforce verifies it, and the tab gets that person's own Salesforce session. No popup, no click, no password.

```
Teams tab --getAuthToken()--> Entra ID issues a token for YOUR SSO app (silent)
Teams tab --POST /services/oauth2/token (token-exchange)--> Salesforce
Salesforce runs SalesforceInTeamsTokenExchangeHandler (Apex):
  1. fetch Microsoft's public signing keys  2. verify the token signature
  3. check issuer / audience / expiry        4. find User by Azure_AD_Email_Id__c
Salesforce returns a real access token for that user -> tab loads their data
```

The popup sign-in stays as the automatic fallback

If any part of this setup is missing or fails for a person, the tab shows a "Sign in" button instead and uses the popup flow. Nothing breaks, so you can roll this out and fix people one at a time. To switch SSO off later, untick "Enable Token Exchange Flow" on the app (Step 2.2) - no code change.

Who is needed

Role	What they do	Steps
Microsoft Entra admin	Registers the SSO app and exposes its API in the client's tenant	Phase 1
Salesforce admin	Deploys metadata, configures the External Client App, permission set and token handler	Phase 2
Developer	Edits the tenant/app ids in code, deploys the relay, builds the Teams package	2.1, 3.1 - 3.3
Teams admin	Removes the old app and installs the new package	3.3

2. Before you start

SSO reuses the same Salesforce app as the popup sign-in, so that sign-in has to exist first.

Prerequisite	Why / how to confirm
Popup sign-in (PKCE) already works	SSO uses the same External Client App (Salesforce_In_Teams) and falls back to it. Confirm the tab loads records after clicking Sign in. If not, set that up first - see "Salesforce-in-Teams-Authentication-PKCE-and-SSO.pdf", Part A.
User.Azure_AD_Email_Id__c exists and is filled in	The handler matches the token's sign-in name (UPN) to this field. It must equal the email/UPN the person signs in to Teams with, exactly. Field metadata: force-app/.../objects/User/fields/.
Relay Function App with a stable hostname	The hostname is baked into the Application ID URI (Step 1.2). If it changes later, Steps 1.2, 2.1 and 3.1 must be redone.
Salesforce CLI logged in to the target org	Used to deploy metadata. Do a sandbox first; Permitted Users (Step 2.3) is an app-wide change.
Teams: custom app upload allowed	The Teams admin must be able to remove and re-upload the app package.

3. Values to collect

These values appear in several places. Write the client's values in the blank column now; the steps below refer to them by name. The last column is what the dev environment used, for reference only.

Name	Where to find it	Client value	Dev example
TENANT_ID	Entra admin center > Overview > Tenant ID		80040ab0-138a-4eaa-8a1c-68e4f0465151
RELAY_HOST	Function App URL (host only, no https://)		case-swarm-relay-sfchatsync-h9hbhzf9eagdbkd6.canadacentral-01.azurewebsites.net
SSO_APP_CLIENT_ID	Created in Step 1.1 (Application ID)		b186fa34-7fdf-48be-80c3-7e736eb501b3
APP_ID_URI	Built in Step 1.2: api://RELAY_HOST/SSO_APP_CLIENT_ID		api://<RELAY_HOST>/b186fa34-...b3
SF_MY_DOMAIN	Salesforce My Domain host		mylightningapp-dev-dev-ed.my.salesforce.com
SF_CONSUMER_KEY	External Client App > Consumer Key		3MVG9G9pzCUSkzZsud2BdW9T... (public client, not secret)
TEAMS_APP_ID	manifest.json top-level id (unchanged)		3aed85f6-5208-454a-8c48-9ad5fbd14057

4. Setup steps

Phase 1 - Microsoft Entra ID (the client's tenant)

Done by: Entra admin

[] 1.1 Register the SSO app

- Entra admin center > App registrations > New registration.
- Name: Salesforce In Teams SSO. Supported account types: Single tenant only.
- Redirect URI: leave blank (getToken() does no redirect).
- After Register, copy the Application (client) ID = SSO_APP_CLIENT_ID and the Directory (tenant) ID = TENANT_ID.

Check: The app appears under App registrations; you have both ids written down.

[] 1.2 Set the Application ID URI

- Expose an API > Application ID URI > Add / Edit.
- Set it to exactly: api://RELAY_HOST/SSO_APP_CLIENT_ID
- The hostname is mandatory. A bare api://SSO_APP_CLIENT_ID fails in Teams with "App resource defined in manifest and iframe origin do not match".

Check: The saved value shows the relay hostname between api:// and the client id.

[] 1.3 Add the access scope

- Expose an API > Add a scope.
- Scope name: access_as_user. Who can consent: Admins and users. State: Enabled.
- Consent display name/description: any clear text, e.g. "Access Salesforce as the signed-in user".

Check: The scope is listed as api://.../access_as_user, State = Enabled.

[] 1.4 Pre-authorize the two Teams clients

- Expose an API > Add a client application - do this twice, ticking the access_as_user scope each time:
- 1fec8e78-bce4-4aaf-ab1b-5451cc387264 (Teams desktop and mobile)
- 5e3ce6c0-2b1f-4285-8d4b-75ee78787346 (Teams web)
- These are Microsoft's fixed client ids for Teams. Without them Teams cannot issue the token silently.

Check: Both ids are listed under Authorized client applications with the scope ticked.

[] 1.5 Force v2.0 tokens

- Manifest blade > in the api object set "requestedAccessTokenVersion": 2 > Save.
- (Older manifest format and the portal label call this accessTokenAcceptedVersion. Same setting.)
- The Apex handler only accepts the v2.0 issuer format; a v1.0 token is rejected.

Check: Later, a real token's issuer is https://login.microsoftonline.com/TENANT_ID/v2.0.

Phase 2 - Salesforce org

Done by: Developer for 2.1, Salesforce admin for the rest

[] 2.1 Put the client's ids into the Apex handler, then deploy

- Edit force-app/main/default/classes/SalesforceInTeamsTokenExchangeHandler.cls - two constants:

```
private static final String TENANT_ID = '<TENANT_ID>';
private static final Set<String> EXPECTED_AUDIENCES = new Set<String>{
    'api://<RELAY_HOST>/<SSO_APP_CLIENT_ID>',    // the Application ID URI
    '<SSO_APP_CLIENT_ID>'                      // a real token's aud carries this bare id
};
```

[] **Deploy these three, in this order (the handler metadata references the class):**

- sf project deploy start -d force-app/main/default/classes/SalesforceInTeamsTokenExchangeHandler.cls
- sf project deploy start -d force-app/main/default/remoteSiteSettings/Azure_AD_JWKS.remoteSite-meta.xml
- sf project deploy start -d force-app/main/default/oauthtokenexchangehandlers
- The Remote Site Setting allows https://login.microsoftonline.com. Without it Apex cannot download Microsoft's signing keys ("Unauthorized endpoint" in the debug log).

Check: Setup > Token Exchange Handlers lists "Salesforce In Teams Token Exchange" as Enabled, type Access Token.

[] **2.2 Enable Token Exchange on the External Client App**

- Setup > External Client App Manager > Salesforce_In_Teams > Policies > "OAuth Flows and External Client App Enhancements" > tick Enable Token Exchange Flow.
- Or deploy the two files that carry it: isTokenExchangeEnabled=true in extlCIntAppGlobalOAuthSets/Salesforce_In_Teams_GlobalOAuth..., isTokenExchangeFlowEnabled=true in extlCIntAppOAuthPolicies/...defaultPolicy.
- Use the client org's own app name if it differs from Salesforce_In_Teams.

Check: The checkbox is ticked. Calling the token endpoint now returns "token handler not found" (fixed in 2.4), not an unsupported-grant error.

[] **2.3 Permitted Users = admin approved, plus a permission set**

- Create a Permission Set named Salesforce_In_Teams (no permissions needed inside it).
- On the same app's Policies page set Permitted Users to "Admin approved users are pre-authorized" and select that permission set.
- In metadata this is permittedUsersPolicyType = AdminApprovedPreAuthorized and commaSeparatedPermissionSet = Salesforce_In_Teams (see the retrieved policy file in the repo).
- Why: token exchange has no consent screen, so Salesforce needs a prior approval on file. Without this the token endpoint returns invalid_grant "user hasn't approved this consumer".
- This setting is app-wide: it also applies to the popup fallback. From now on every person needs the permission set.

Check: Policies page shows the mode and the permission set.

[] **2.4 Link the handler to the app (Setup UI only)**

- Setup > Quick Find "Token Exchange Handlers" > open Salesforce In Teams Token Exchange > Enable New App.
- Select App: Salesforce_In_Teams. Run as: an active user (admin is fine) - the handler's Apex runs as this user to look up Users.
- Tick "Make ... the default handler for this app" > Done.
- There is no deployable metadata for this link; it must be clicked in every org.

Check: The handler's Enabled Apps list shows Salesforce_In_Teams with the Run As user.

[] **2.5 Assign people**

- Assign the Salesforce_In_Teams permission set to every person who will use the tab.
- Set each person's User.Azure_AD_Email_Id__c to their Entra sign-in name (UPN), e.g. first.last@clientdomain.com.

Check: Each user has the permission set and a populated Azure_AD_Email_Id__c.

Phase 3 - Teams app and relay

Done by: Developer, then Teams admin for 3.3

[] 3.1 Update the Teams manifest

- In teamsapp-salesforce/manifest.json add or update webApplicationInfo (see below).
- validDomains must include RELAY_HOST and SF_MY_DOMAIN. Increase the version number - Teams rejects a re-upload with the same version.

```
"webApplicationInfo": {  
  "id": "<SSO_APP_CLIENT_ID>",  
  "resource": "api://<RELAY_HOST>/<SSO_APP_CLIENT_ID>"    <- same as APP_ID_URI  
}
```

[] 3.2 Point the page code at the client's org, then deploy the relay

- In relay/src/functions/salesforceTabHome.js and salesforceAuthCallback.js set SF_LOGIN_DOMAIN (https://SF_MY_DOMAIN), SF_CLIENT_ID (SF_CONSUMER_KEY) and SF_REDIRECT_URI (https://RELAY_HOST/api/salesforceAuthCallback). The two files must agree.
- Deploy: func azure functionapp publish <function-app-name> (run inside relay/).

Check: Opening https://RELAY_HOST/api/salesforceTabHome in a browser returns the page (it will ask to sign in).

[] 3.3 Install the package fresh

- Zip manifest.json + color.png + outline.png (files at the zip root, not in a folder).
- Teams admin: fully REMOVE the old app first, do not just update it. Teams keeps a stale cached manifest when webApplicationInfo changes and reports "App webApplicationInfo or resource not defined in manifest".
- Upload the new package as a fresh install. Test in Teams on the web first (it caches less than the desktop app); fully quit and reopen Teams desktop before testing there.

Check: The tab opens from Teams with the new version number.

Phase 4 - Verify

Done by: Anyone with the permission set

[] 4.1 Open the tab as a person set up in Step 2.5

- Expected: the tab loads their records with no popup and no "Sign in" button.
- Open browser dev tools > Console. There must be no "Teams SSO sign-in failed" line.
- If a Sign in button appears instead, silent SSO failed for that person and the popup fallback is doing its job. The console line names the reason - look it up below.

5. Error decoder

Match the exact message in the browser console (or the Network tab response of the failed POST to /services/oauth2/token).

What you see	Cause	Fix
App resource defined in manifest and iframe origin do not match	Application ID URI has no hostname, or manifest resource differs from it	Step 1.2, then make manifest resource identical (Step 3.1)
App webApplicationInfo or resource not defined in manifest	Teams is running an old cached manifest	Step 3.3: remove the app, reinstall fresh, test on web first
AADSTS500011 ... resource principal ... not found in the tenant	The URI Teams requests does not exist in the signed-in person's tenant (typo, unsaved, or app registered in a different tenant)	Compare manifest resource with Entra > Expose an API; confirm the app is in TENANT_ID
FailedToOpenWindow	Browser blocked an automatic popup	Expected when SSO fails: click the Sign in button (popup opens only on click)
invalid_request: token handler not found	Handler not linked to the app	Step 2.4 (Enable New App)
invalid_request: token handler validation failed	Apex rejected the token. The real reason is only in the debug log	Read the log (recipe below). Usual: wrong TENANT_ID/audience in the class, missing Remote Site Setting, v1 token
invalid_grant: user hasn't approved this consumer	Permitted Users is not admin-approved, or the person lacks the permission set	Steps 2.3 and 2.5
Debug log: Unauthorized endpoint ... Remote site settings	Apex may not call login.microsoftonline.com	Deploy Azure_AD_JWKS (Step 2.1)
SSO works for some people, others get the Sign in button	That person has no matching Azure_AD_Email_Id__c (token handler validation failed) or no permission set (invalid_grant)	Check Azure_AD_Email_Id__c equals their UPN and the permission set is assigned (Step 2.5)

Reading the real rejection reason (debug log recipe)

Salesforce never tells the browser why a custom handler rejected a token. To see it, trace the handler's Run As user for a few hours, have the person retry, then read the log. Add a temporary System.debug in the handler's catch block if you need the exception text.

```
# 1. trace flag (Tooling API), 3 hours; DebugLevel id from: SELECT Id FROM DebugLevel
sf data create record --subject TraceFlag --use-tooling-api --values \
  "TracedEntityId=<RUN_AS_USER_ID> DebugLevelId=<ID> LogType=DEVELOPER_LOG \
  StartDate=<now UTC> ExpirationDate=<now+3h UTC>"

# 2. after a retry, find the log
sf data query --use-tooling-api -q "SELECT Id,StartTime FROM ApexLog \
  WHERE Operation='OAuthTokenExchangeApexExec' ORDER BY StartTime DESC LIMIT 3"

# 3. download it: GET /services/data/v61.0/tooling/subjects/ApexLog/<Id>/Body
```

6. Client considerations

Verified vs not verified

Verified in the dev environment	NOT verified - test in the client before relying on it
Single Entra tenant; Teams on the web; silent sign-in end to end with a real token	Teams desktop and mobile silent sign-in (uses the other pre-authorized client id)
Salesforce Developer Edition org, External Client App with Token Exchange	Tenants with Conditional Access or restricted user consent (a consent error may need tenant admin consent - not seen here)
Fallback to the popup when SSO fails	Guest / external users, or people whose Teams tenant differs from TENANT_ID (their token will not match the issuer check)
Real token: iss = login.microsoftonline.com/TENANT/v2.0, aud = bare client id	Sovereign clouds (GCC High, China): different login hosts than login.microsoftonline.com
Popup fallback shows Salesforce's own login page	A client whose Salesforce login is Okta: token exchange never uses the Salesforce login, so it should be unaffected, but only the popup fallback will show Okta

Change management

- Permitted Users (Step 2.3) applies to the whole app and to the popup fallback: every user needs the permission set. Plan the rollout before flipping it in production.
- The Remote Site Setting adds one allowed outbound host: Microsoft's public login endpoint (signing keys only, no credentials sent).
- The handler is the security boundary: it does full signature verification plus issuer, audience and expiry checks. Do not weaken EXPECTED_AUDIENCES or the issuer check.
- Each silent sign-in makes one outbound callout for Microsoft's keys - well inside Salesforce callout limits.
- Rollback: untick Enable Token Exchange Flow (Step 2.2). Everyone falls back to the popup on their next load; no redeploy needed.

7. One-page checklist

Done	Step	Action
[]	Prereq	Popup sign-in works; Azure_AD_Email_Id__c field exists; relay hostname is final
[]	1.1	Entra: register "Salesforce In Teams SSO" (single tenant, no redirect URI); note client id + tenant id
[]	1.2	Application ID URI = api://RELAY_HOST/SSO_APP_CLIENT_ID
[]	1.3	Scope access_as_user (admins and users, enabled)
[]	1.4	Authorize 1fec8e78-...387264 and 5e3ce6c0-...346 on that scope
[]	1.5	api.requestedAccessTokenVersion = 2
[]	2.1	Edit TENANT_ID + EXPECTED_AUDIENCES in the Apex class; deploy class, Azure_AD_JWKS, token handler
[]	2.2	Enable Token Exchange Flow on the External Client App
[]	2.3	Permission set Salesforce_In_Teams; Permitted Users = admin approved (AdminApprovedPreAuthorized)
[]	2.4	Token Exchange Handlers > Enable New App: app + Run As user + default handler
[]	2.5	Assign permission set; fill Azure_AD_Email_Id__c = UPN for each user
[]	3.1	manifest.json webApplicationInfo id/resource, validDomains, version bumped
[]	3.2	Page code constants match the client org; func azure functionapp publish
[]	3.3	Remove old Teams app fully; upload new package fresh; test on Teams web first
[]	4.1	Open the tab as a test user: no popup, no Sign in button, no SSO error in console